Imbaraga – Final Project Outline & 2-Week Implementation Plan

AI-powered poverty early-warning and social-protection targeting platform for Rwanda

Version: Final v1.0 | **Build window:** 14 days | **Stack:** React + TypeScript + Tailwind | Node.js + Express REST API | Python (pandas, scikit-learn, XGBoost, SHAP) ML service

---

1. Executive Summary

Imbaraga (Kinyarwanda: "strength / power") is a decision-support platform that predicts which households and districts are at risk of falling into poverty *between* official survey and Ubudehe cycles, explains *why* in plain language (English + Kinyarwanda), and lets program officers rank households and export targeting lists in minutes.

Item	Detail
Project name	Imbaraga
One-liner	Predicts households at risk of falling into poverty and helps programs like VUP reach them faster
Target users	(1) Social protection officers (LODA, MINALOC, NGOs), (2) Policymakers (district and national planners), (3) Field and community leaders
Alignment	NST2 – poverty reduction and social protection pillar; Vision 2050 – inclusive growth and quality of life
Nature of product	Targeting and decision-support layer only. No money movement, no personal data collection

The three things the judges must remember:

1. A working risk model with honest validation (baseline vs XGBoost, district-aware split).
2. Every score is explainable (top 3 drivers in plain language).
3. A live map showing where poverty risk and financial exclusion overlap ("double exclusion").

---

2. Problem Statement

Rwanda measures poverty through periodic surveys (EICV roughly every 3–5 years) and Ubudehe categorization cycles. Households that fall into poverty between cycles are invisible to the system, so cash transfers, health insurance subsidies and financial services arrive late or miss them entirely.

#	Sub-problem	What it means	Feature that solves it
P1	Stale targeting	Ubudehe categories and VUP lists do not reflect recent shocks (job loss, crop failure, illness)	Household risk scoring (F1, F5)
P2	Double exclusion	Households with no mobile money, savings or credit in high-poverty districts are hit hardest and recover slowest	Risk × financial-access map (F3)
P3	No early warning for planners	LODA and districts react after statistics are published	District dashboard and insights (F3, F4)
P4	No transparency in ranking	Officers cannot see why a household is flagged	SHAP explanations (F2)

Evidence to prepare before pitching (download and cite exactly, do not use remembered numbers):

- National and district poverty rates from the latest NISR EICV report.
- Financial exclusion percentages from FinScope Rwanda (national and, if available, by district or province).
- Ubudehe category distribution and its update cycle.
- WFP/NISR CFSVA food-insecurity figures for context overlay.

> **Verify early:** the original brief labels the dataset “EICV 5 (2022/23)”. EICV5 is commonly cited as 2016/17, and a newer EICV round may exist. Confirm the exact edition, its variables and whether microdata access has been granted on the NISR microdata portal. This decides whether you train on real microdata or on the fallback described in Section 9.5.

---

3. Objectives and Success Criteria

#	Objective	Measurable MVP success criterion	How to prove it in the demo
O1	Predict household and district poverty risk	AUC ⩾ 0.80 and accuracy ⩾ 75% on held-out test set	Metrics card on Insights page and a slide with confusion matrix and ROC curve
O2	Explain each score	Every scored household shows top 3 drivers in plain language, in EN and RW	Click a row and show the reason chips
O3	Identify double-exclusion zones	District map renders live with a toggle between poverty risk and financial access gap	Live map interaction
O4	Speed up targeting	Upload up to 200 households, score, rank and export CSV in under 5 minutes (target under 1 minute)	Timed live demo
O5	Be usable and presentable	Three polished pages, English/Kinyarwanda toggle, responsive on laptop and tablet	Walkthrough

---

4. Scope

4.1 In scope (MVP – what we WILL build)

- ML poverty-risk classifier (XGBoost primary, Logistic Regression baseline).
- Explainability layer: SHAP values mapped to plain-language sentences in English and Kinyarwanda.
- District-level interactive dashboard: risk heatmap and financial-access gap.
- Targeting Assistant: upload CSV or enter rows, score, rank, export CSV.
- Data pipeline: cleaning, encoding Ubudehe categories, merging NISR and FinScope aggregates.

- Kinyarwanda/English toggle (at minimum all key labels, KPI names, page titles, risk levels and driver sentences).
- Demo dataset of 50–200 anonymized, synthetic-but-realistic households.

4.2 Out of scope (state this confidently as roadmap)

- Direct cash disbursement. We are a targeting and decision-support layer only.
- Individual citizen mobile app (farmer-facing version is future work).
- Real-time feeds from government registries (requires MOUs with LODA/MINALOC).
- Collection of personally identifiable data. We use aggregated or anonymized survey microdata only, consistent with Law No. 058/2021 on personal data protection.
- Live SMS/USSD alerts (shown only on an architecture slide).
- Automated retraining and MLOps (static trained model for MVP).
- Province-level policy prescriptions. We present risk scores, not policy.
- Mobile money or fintech integration.

---

5. Users and Key User Stories

Persona	Story	Acceptance test
Program officer (LODA / NGO)	As an officer, I upload a household list and see who is most at risk so I can prioritize VUP support	CSV of 200 rows returns ranked list with risk level and top 3 reasons, exportable
District planner	As a planner, I want to see which sectors/districts combine high poverty risk with low financial access	Map highlights double-exclusion districts and lists the top 5
Policymaker	As a national planner, I want KPI summaries and drivers of risk so I can brief leadership	Dashboard KPIs and driver chart load in under 3 seconds
Field / community leader	As a field leader, I want a simple explanation in Kinyarwanda of why a household is flagged	Reason chips display in Kinyarwanda when toggled

---

6. Functional Requirements

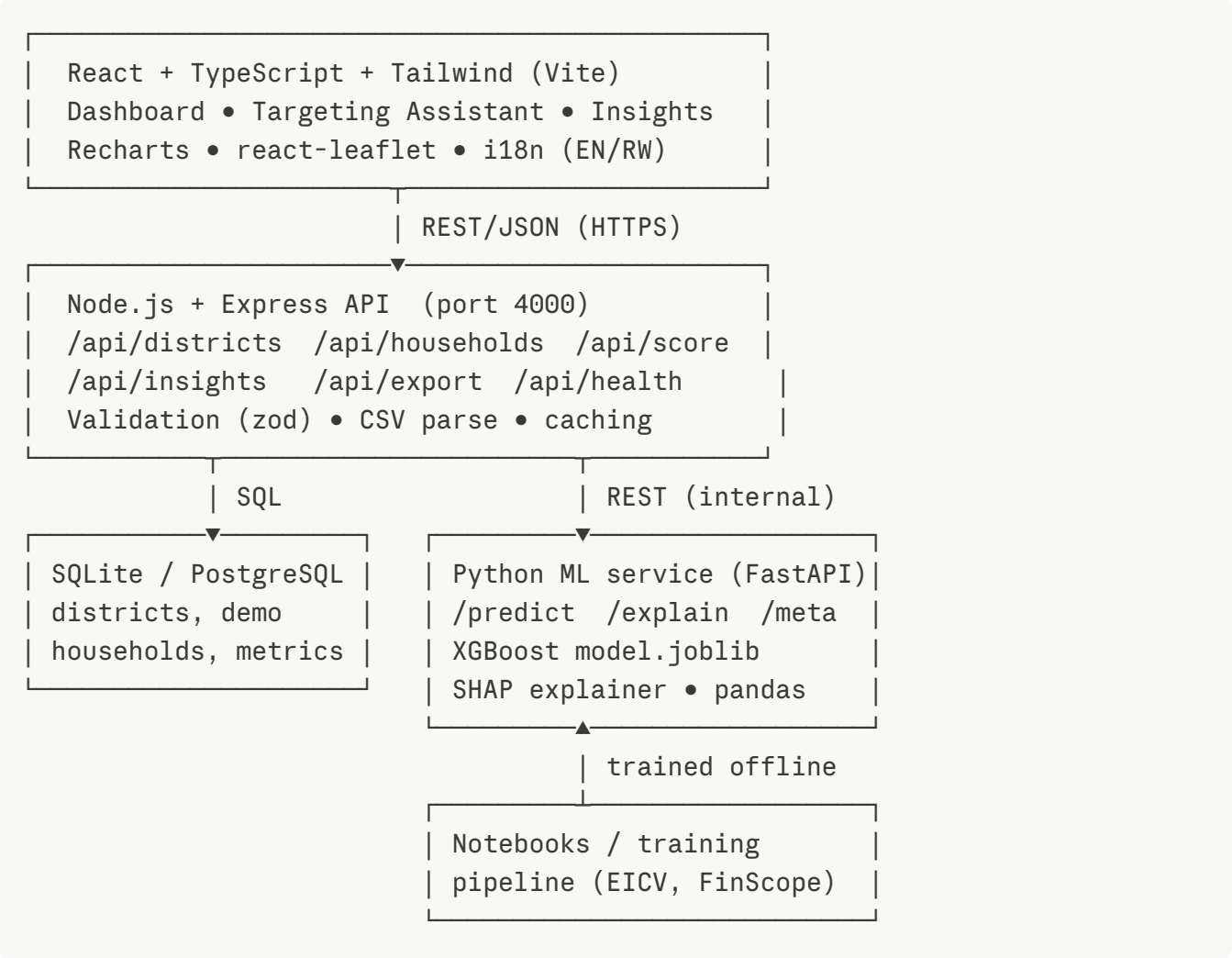
Priority: **M** = Must (demo fails without it), **S** = Should, **C** = Could (only if time remains).

ID	Feature	Description	Priority
F1	Poverty risk model	Binary classifier outputs probability (0–1) and risk level: Low < 0.33, Medium 0.33–0.66, High > 0.66 (thresholds tunable)	M
F2	Explainability	Top 3 SHAP drivers per household mapped to sentences, EN and RW	M
F3	District dashboard	Choropleth map of 30 districts, KPI cards, risk × financial-access scatter, top drivers bar chart	M
F4	Insights page	District deep-dive: risk distribution, exclusion breakdown, global SHAP importance, model metrics	M
F5	Targeting Assistant	Table of households, CSV upload, Score All button, sorting and filtering, reason chips, CSV export	M
F6	Language toggle	EN/RW switch persisted in local storage	M
F7	Demo data generator	Script producing 50–200 synthetic households that look realistic and produce interesting results	M
F8	Fairness view	Simple chart of average predicted risk and flag rate by sex of household head and by district	S
F9	What-if scenario	Adjust a household's shock flags (e.g. job loss) and see the score change	C
F10	Printable summary	Print-friendly district brief	C

---

7. System Architecture

The system has three tiers. The **React frontend** talks only to the **Node/Express API**. Express owns the database, validation, CSV handling and caching, and calls the **Python ML service** over REST for predictions and explanations. This keeps the web layer in the stack you chose while keeping all data science in Python.



Request flow for "Score" in the Targeting Assistant:

1. User uploads CSV → frontend sends rows to `POST /api/score`.
2. Express validates columns and types (zod), fills defaults, and forwards the batch to the Python service `POST /predict`.
3. Python returns probability, risk level and top 3 drivers (feature, direction, SHAP value) per row.
4. Express stores results, translates driver codes to sentences using a shared mapping, and returns the ranked list.
5. Frontend renders table, chips and enables CSV export.

---

8. Technology Stack

Layer	Technology	Notes
Frontend	React 18, TypeScript, Vite	Component-based, typed API client
Styling	Tailwind CSS	Design tokens for risk colors (green / amber / red)
Charts	Recharts	Bar, scatter, line, area, radar, pie as needed
Map	react-leaflet and Rwanda districts GeoJSON	Obtain from NISR or open sources such as geoBoundaries or HDX; simplify geometry for speed
State and data fetching	TanStack Query, React Router	Caching and loading states
i18n	react-i18next	`en.json` and `rw.json` files
Tables	TanStack Table	Sorting, filtering, pagination
Backend API	Node.js 20, Express, TypeScript	REST, zod validation, multer (uploads), csv-parse / csv-stringify, cors, helmet, morgan
Database	SQLite (better-sqlite3) for demo; PostgreSQL (Supabase) optional	Aggregated indicators and demo households only
ML service	Python 3.11, FastAPI, uvicorn	Thin REST wrapper around the trained model
Data and ML	pandas, numpy, scikit-learn, XGBoost, SHAP, imbalanced-learn (optional), joblib, matplotlib	Training notebooks in Colab/Jupyter
Testing	Vitest and React Testing Library (frontend), Jest and Supertest (API), pytest (ML)	Focus on critical paths only
Deployment	Vercel (frontend), Render or Railway (Express and Python services)	Free tiers; keep a local Docker Compose fallback

Layer	Technology	Notes
Collaboration	GitHub, feature branches, pull requests, Kanban board	Commit daily

---

9. Data and ML Methodology

9.1 Data sources

Source	Used for	Type
NISR EICV (latest available round)	Household features and poverty labels (training)	NISR microdata
NISR district poverty reports	District validation and map values	NISR published tables
FinScope Rwanda	Financial access: mobile money, savings, credit (district or province aggregates)	External survey
World Bank High-Frequency Phone Survey	Shock dynamics (job loss, food insecurity); used for feature design and context	External
CFSVA (WFP/NISR)	Food-security overlay	WFP/NISR

9.2 Pipeline

1. **Ingest:** load EICV household and consumption files; FinScope aggregates by district/province.
2. **Clean:** handle missing values (median or mode, plus missing flags), remove duplicates, standardize district codes and names (create one master district lookup of 30 districts).
3. **Label:** binary `poor` = 1 if consumption per adult equivalent is below the national poverty line; 0 otherwise. Save the line value used.
4. **Feature engineering:** see Section 9.3.

5. **Split:** 80/20 train/test stratified on label and district. Also run **GroupKFold by district** to check generalization to unseen districts.
6. **Train:** Logistic Regression baseline (scaled features) and XGBoost (primary). Tune with a small randomized search.
7. **Imbalance:** use `scale_pos_weight` for XGBoost and `class_weight='balanced'` for logistic regression. Report precision and recall for the poor class, not only accuracy.
8. **Evaluate:** accuracy, precision, recall, F1, ROC-AUC, PR-AUC, confusion matrix, calibration curve.
9. **Explain:** SHAP TreeExplainer; per-household top 3 drivers; global importance chart.
10. **Export:** `model.joblib`, `feature_schema.json`, `metrics.json`, `district_aggregates.csv`.

> **Avoid label leakage:** the label comes from consumption, so do **not** include consumption, expenditure or food-spending variables as features. Use only structural and observable proxies (demographics, assets, housing, land, income sources, access, shocks). Say this in the pitch; it shows rigor.

9.3 Feature set (map each to the actual EICV variable name during Day 2)

Group	Feature	Type
Demographics	household_size, dependency_ratio, head_age, head_sex, head_education_level, female_headed, num_children_under_5, has_disabled_member	numeric / categorical
Economic	main_income_source, income_sources_count (diversity), has_wage_income, land_size_ha, livestock_index	numeric / categorical
Assets and housing	asset_index (PCA or weighted count), roof_material, floor_material, has_electricity, improved_water, improved_sanitation, cooking_fuel	numeric / binary
Social	ubudehe_category (ordinal 1–4), receives_vup (if available)	ordinal / binary

Group	Feature	Type
Financial access	has_mobile_money, has_savings, has_credit, financial_access_score (0–3)	binary / numeric
Shocks (if available)	shock_job_loss, shock_crop_failure, shock_illness	binary
Geography	district_code, rural_urban, distance_to_market, district-level aggregates (FinScope access rate, district poverty rate)	mixed

9.4 Explanation mapping (SHAP → plain language)

Each feature has a bilingual sentence template for “raises risk” and “lowers risk”. Example entries (store in `explanations.json`, shared by the ML and API layers):

Feature	Direction	English	Kinyarwanda (have a native speaker review)
has_mobile_money	raises risk when 0	No mobile money account limits access to savings and transfers	Nta konti ya mobile money ifite bituma kubona ubwizigame n’inkunga bigorana
dependency_ratio	raises risk when high	Many dependents per working adult	Abatunzwe ni benshi ugereranyije n’abakora
land_size_ha	raises risk when low	Very small land holding	Ubutaka buto cyane
income_sources_count	raises risk when 1	Relies on only one source of income	Yishingikiriza isoko imwe y’amafaranga gusa
shock_job_loss	raises risk when 1	Recent job loss in the household	Gutakaza akazi vuba aha mu rugo

Rule: show the top 3 features with the largest positive SHAP value (those that push toward poverty) as red chips. Optionally show one protective factor as a green chip.

9.5 Data-access fallback plan (decide by Day 2 end)

Scenario	Action
Real EICV microdata obtained	Train on it; report real metrics
Microdata delayed or restricted	Use publicly available survey data with similar structure (for example DHS Rwanda household data, if accessible) for training, **or** generate a synthetic training set calibrated to published NISR district distributions. Label clearly as synthetic and never claim real-world accuracy
Neither	Synthetic data only; present the pipeline and architecture as the contribution and be transparent

In all cases the demo households are synthetic and anonymized.

---

10. REST API Specification

10.1 Express API (public to the frontend), base path `/api`

Method	Endpoint	Purpose	Key response fields
GET	`/api/health`	Health check of API and ML service	status, mlService, modelVersion
GET	`/api/districts`	All 30 districts with risk and access indicators	code, name, povertyRate, riskScore, financialAccessRate, exclusionGap, doubleExclusionFlag
GET	`/api/districts/:code`	One district deep-dive	indicators, riskDistribution, topDrivers, trend
GET	`/api/kpis`	National KPI cards	povertyPct, exclusionPct, highRiskDistrictCount, householdsScored

Method	Endpoint	Purpose	Key response fields
GET	`/api/households`	List demo households (paginated, filter by district and risk)	rows, total
POST	`/api/households`	Add one household row (manual entry)	id
POST	`/api/score`	Score a batch of households (JSON or CSV upload)	per row: id, probability, riskLevel, drivers[3], rank
GET	`/api/export/:batchId`	Download scored batch as CSV	CSV file
GET	`/api/insights/drivers`	Global SHAP feature importance	feature, label (EN/RW), meanAbsShap
GET	`/api/insights/metrics`	Model metrics for both models	accuracy, precision, recall, f1, auc, confusionMatrix, rocPoints
GET	`/api/insights/fairness`	Flag rate and average risk by group	group, value, avgRisk, flagRate
GET	`/api/insights/scatter`	District points for risk × financial access chart	code, name, risk, access

Standard conventions: JSON responses, `{ error: { code, message } }` on failure, HTTP 400 for validation errors, 503 if the ML service is down, request size limit of 2 MB, CORS restricted to the frontend origin.

10.2 Python ML service (internal), base path `/`

Method	Endpoint	Purpose
GET	`/health`	Liveness and model version
GET	`/meta`	Feature schema, thresholds, global importance, metrics

Method	Endpoint	Purpose
POST	`/predict`	Body: `{ households: [ {...features} ] }` → probability, risk level and top 3 drivers per household

Example `/predict` response item:

```
{
  "id": "HH-0042",
  "probability": 0.81,
  "riskLevel": "High",
  "drivers": [
    { "feature": "has_mobile_money", "value": 0, "shap": 0.21, "direction":
"raises" },
    { "feature": "dependency_ratio", "value": 2.5, "shap": 0.17, "direction":
"raises" },
    { "feature": "land_size_ha", "value": 0.1, "shap": 0.12, "direction":
"raises" }
  ]
}
```

10.3 CSV upload template for the Targeting Assistant

Required columns: `household_id, district_code, household_size, dependency_ratio, head_age, head_sex, head_education_level, ubudehe_category, main_income_source, income_sources_count, land_size_ha, asset_index, has_electricity, improved_water, has_mobile_money, has_savings, has_credit`. Optional: shock flags. Provide a downloadable template file and clear row-level error messages (for example “Row 14: ubudehe_category must be 1–4”). Missing optional values are filled with training medians.

---

11. Database Design

The database holds only aggregated indicators and synthetic households, never real personal data.

Table	Key columns
`districts`	code (PK), name_en, name_rw, province, population, poverty_rate, financial_access_rate, exclusion_gap, risk_score, double_exclusion (bool)
`households_demo`	id (PK), district_code (FK), all model features, created_at
`score_batches`	id (PK), created_at, row_count, model_version
`scores`	id (PK), batch_id (FK), household_id, probability, risk_level, driver1, driver2, driver3, rank
`model_metrics`	model_name, accuracy, precision, recall, f1, auc, created_at
`feature_importance`	feature, label_en, label_rw, mean_abs_shap

Double-exclusion rule (define it once and reuse everywhere): a district is flagged when its risk score is in the top tercile **and** its financial access rate is in the bottom tercile. Show the rule on the dashboard so judges can see it is transparent.

---

12. Frontend Design

12.1 Pages (keep to three)

Page	Route	Contents
Dashboard	`/`	Rwanda risk heatmap (toggle: poverty risk / financial access gap / double exclusion); KPI cards (poverty %, exclusion %, high-risk districts, households scored); scatter chart (risk × financial access); top drivers bar chart; top 5 priority districts list
Targeting Assistant	`/targeting`	CSV upload and manual add; editable table; Score All button; risk column with colored badge; reason chips (EN/RW); sort and filter by district and risk level; Download CSV; row drawer with per-household SHAP bar chart

Page	Route	Contents
Insights	`/insights`	District selector; risk distribution histogram; exclusion breakdown (mobile money, savings, credit); global SHAP importance; model comparison (baseline vs XGBoost); ROC curve; confusion matrix; fairness view

12.2 Required charts

#	Chart	Library / type	Page	Data endpoint
C1	District choropleth map	react-leaflet GeoJSON	Dashboard	`/api/districts`
C2	Risk × financial access scatter (bubble = population, color = double-exclusion flag)	Recharts ScatterChart	Dashboard	`/api/insights/scatter`
C3	Top risk drivers (horizontal bar)	Recharts BarChart	Dashboard, Insights	`/api/insights/drivers`
C4	KPI sparklines or mini cards	Recharts / Tailwind	Dashboard	`/api/kpis`
C5	Per-household SHAP contribution bars	Recharts BarChart (diverging)	Targeting drawer	`/api/score`
C6	Risk level distribution (donut or stacked bar)	Recharts PieChart / BarChart	Targeting, Insights	`/api/score`, `/api/districts/:code`
C7	Exclusion breakdown by district (grouped bar)	Recharts BarChart	Insights	`/api/districts/:code`

#	Chart	Library / type	Page	Data endpoint
C8	ROC curve and baseline vs XGBoost	Recharts LineChart	Insights	`/api/insights/metrics`
C9	Confusion matrix	Tailwind grid heatmap	Insights	`/api/insights/metrics`
C10	Fairness comparison by group	Recharts BarChart	Insights	`/api/insights/fairness`
C11	District risk trend (optional, only if time-series data exist)	Recharts LineChart	Insights	`/api/districts/:code`

Chart rules: always label axes and units, use one consistent risk color scale (green → amber → red), include tooltips, show loading skeletons and empty states, and do not rely on color alone (add text or icons).

12.3 Component structure

```
src/
  api/            typed client + TanStack Query hooks
  components/
    layout/       Navbar, LanguageToggle, PageShell
    charts/       RiskMap, ScatterRiskAccess, DriversBar, ShapBar, RocCurve,
ConfusionMatrix
    ui/           KpiCard, RiskBadge, ReasonChip, DataTable, Drawer,
Skeleton, ErrorState
  pages/          Dashboard, Targeting, Insights
  i18n/           en.json, rw.json
  types/          shared TypeScript interfaces
  utils/          formatters, csv helpers, risk color scale
```

12.4 UX and accessibility requirements

- Responsive on laptop and tablet; readable on a projector (large fonts, high contrast).
- Every async view has loading, empty and error states.
- Risk always shown as color **plus** label (Low / Medium / High).

- Consistent terminology between English and Kinyarwanda; keep a glossary file.
- A visible “Decision-support only. A human makes the final decision” notice on the Targeting page.

---

13. Repository Structure

```
imbaraga/
  README.md
  docker-compose.yml
  frontend/            React + TS + Tailwind (Vite)
  api/                 Node + Express + TS
    src/routes/  src/services/  src/db/  src/validators/
src/shared/explanations.json
  ml/                  Python
    notebooks/         01_eda.ipynb  02_features.ipynb  03_train_eval.ipynb
04_shap.ipynb
    pipeline/          clean.py  features.py  train.py  evaluate.py
explain.py
    service/           main.py (FastAPI)  schemas.py
    artifacts/         model.joblib  feature_schema.json  metrics.json
    data/              raw/ (gitignored)  processed/  synthetic/
  data/geo/            rwanda_districts.geojson
  docs/                architecture.md  api.md  pitch/  data-sources.md
```

Environment variables: `VITE_API_URL`, `PORT`, `ML_SERVICE_URL`, `DATABASE_URL`, `CORS_ORIGIN`, `MODEL_PATH`.

---

14. 14-Day Implementation Plan

The plan assumes a team of 2–4 people split into tracks. If you work alone, follow the same order and skip the items marked Could.

Track	Owner focus
A – Data/ML	Data acquisition, pipeline, model, SHAP, ML service
B – Backend	Express API, database, integration with ML service, CSV export

Track	Owner focus
C – Frontend	React app, charts, map, i18n, UX
D – Pitch/QA (can be shared)	Evidence numbers, slides, testing, demo script, Kinyarwanda review

Week 1 – Foundations and core build

Day	Track A – Data/ML	Track B – Backend	Track C – Frontend	Milestone / exit criteria
1	Request and download EICV, FinScope; create master district lookup; set up repo and Colab	Init Express + TS project, folder structure, health route, SQLite setup	Init Vite + React + TS + Tailwind, routing, layout shell, design tokens	Repos running locally; data access status known
2	Explore data (EDA); define label and feature list; decide fallback (Section 9.5)	Write DB schema and seed script for districts; mock endpoints returning sample JSON	Build Navbar, language toggle, KpiCard, RiskBadge; set up TanStack Query and typed API client	**Decision gate:** real data vs synthetic fallback confirmed
3	Cleaning pipeline and feature engineering; baseline logistic regression	Implement `/api/districts`, `/api/kpis` with seeded data	Dashboard layout with KPI cards and placeholder charts; get GeoJSON loaded	Clean processed dataset exists; Dashboard skeleton visible
4	Train XGBoost; handle imbalance; first metrics	Implement upload endpoint, CSV parsing and zod validation	RiskMap choropleth wired to mock `/api/districts`; scatter chart C2	First model metrics recorded; map renders

Day	Track A – Data/ML	Track B – Backend	Track C – Frontend	Milestone / exit criteria
5	SHAP explanations; explanations.json (EN/RW); save artifacts	Build FastAPI-facing client in Express; `/api/score` contract finalized with mock fallback	Targeting page: table, upload, Score button (mock data)	SHAP top-3 working in notebook
6	Build FastAPI service `/predict`, `/meta`; test with curl	Connect Express to real ML service; store scores; `/api/export`	Connect Targeting page to real API; reason chips; CSV download	**End-to-end scoring works** (upload → score → export)
7	District aggregates and risk scores; double-exclusion computation; synthetic demo households (F7)	Serve real district data; `/api/insights/*` endpoints	Wire Dashboard to real data; drivers bar chart C3	**Checkpoint 1:** Dashboard + Targeting working on real model output. Buffer / catch-up half-day

Week 2 – Completion, polish and pitch readiness

Day	Track A – Data/ML	Track B – Backend	Track C – Frontend	Milestone / exit criteria
8	Final model tuning; GroupKFold by district; calibration; freeze model	Metrics, ROC points and fairness endpoints	Insights page layout; district selector; charts C6, C7	Model frozen (v1.0); Insights skeleton

Day	Track A – Data/ML	Track B – Backend	Track C – Frontend	Milestone / exit criteria
9	Fairness analysis; write methodology notebook; confusion matrix and ROC data	Caching, error handling, rate/size limits, logging	Insights: SHAP importance, ROC, confusion matrix, fairness charts (C8–C10)	All three pages feature-complete
10	Kinyarwanda translation review with native speaker; finalize explanation sentences	Unit tests for validators and routes	Complete i18n (labels, KPIs, chips); responsive fixes; loading/empty/error states	**Checkpoint 2:** feature freeze at end of day
11	Support QA; prepare metrics slide assets (ROC, importance, confusion matrix images)	Deploy ML service and API (Render/Railway); environment variables	Deploy frontend (Vercel); fix CORS and URLs; performance check	Public demo URL works
12	Prepare answers on model, bias, data	Bug fixes; seed final demo data; backup of database and model artifacts	UI polish: spacing, colors, projector readability; record backup demo video	Full dress rehearsal #1 (timed)
13	Finalize evidence numbers and sources	Smoke tests on deployed stack; add fallback offline/local mode via Docker Compose	Final bug fixes only; screenshot set for slides	Pitch deck complete; rehearsal #2 with Q&A

Day	Track A – Data/ML	Track B – Backend	Track C – Frontend	Milestone / exit criteria
14	**Pitch day.** Final check 2 hours before: all services awake, demo data loaded, backup video and local copy ready			Present

Rules for staying on schedule:

- Day 2 and Day 6 are the critical gates. If either slips, cut Could-priority items immediately (F9, F10, C11).
- Free-tier services sleep: ping them 15 minutes before the demo.
- Feature freeze on Day 10. After that only bug fixes, polish and pitch work.
- Commit and push at least once daily; merge to `main` only when the app runs.

---

15. Testing and Quality Assurance

Area	What to test	Tool
ML	No leakage (no consumption variables in features); schema matches between training and service; predictions in [0,1]; same input gives same output	pytest, notebook checks
API	Validation errors, CSV edge cases (missing columns, wrong types, empty file), ML service down (503), pagination	Jest + Supertest
Frontend	Table sorting/filtering, language toggle, upload flow, chart rendering with empty data	Vitest + React Testing Library, manual checks

Area	What to test	Tool
End-to-end	Upload 200-row CSV → score → export in under 1 minute; dashboard loads in under 3 seconds	Manual with stopwatch
Demo readiness	Full rehearsal on the actual presentation laptop and network; backup plan if internet fails	Checklist in Section 20

Definition of Done for any feature: works end-to-end with real API data, has loading/empty/error states, works in both languages, committed and merged to `main`.

---

16. Deployment Plan

Component	Platform	Notes
Frontend	Vercel	Set `VITE_API_URL` to deployed API
Express API	Render or Railway	Set `ML_SERVICE_URL`, `CORS_ORIGIN`; persistent disk or hosted Postgres if SQLite file must persist
ML service	Render or Railway (Python web service)	Include `model.joblib` in the image; pin library versions in `requirements.txt`
Fallback	Docker Compose on a laptop	One command starts all three services; test it on Day 13
Backup	Screen-recorded demo video and exported slides	Use if network or hosting fails

---

17. Ethics, Privacy and Fairness

- **Privacy:** only aggregated or anonymized survey data and synthetic households. No real household registry. Aligned with Law No. 058/2021 on personal data protection; any future real-data deployment needs formal data-sharing agreements and a data protection impact assessment.

- **Human in the loop:** the model ranks and explains; officers and communities make final decisions. State this on the UI and in the pitch.
- **Fairness:** check flag rates and average risk by sex of household head, rural/urban and district; report findings honestly, including limits.
- **Transparency:** show metrics, the double-exclusion rule, data sources and the model version in the app.
- **Limitations to state openly:** survey data age, risk of district-level bias, synthetic demo data, no causal claims, probability is not certainty.
- **Harm avoidance:** scores are never shown publicly at household level; the export contains only IDs from the uploaded file.

---

18. Risk Register

#	Risk	Likelihood	Impact	Mitigation
R1	EICV microdata not accessible in time	Medium	High	Day 2 decision gate; synthetic/alternative data fallback (Section 9.5); request access on Day 1
R2	Model accuracy below target	Medium	Medium	Report honestly; show baseline vs XGBoost; focus on recall for poor class and explainability
R3	District codes do not match across datasets	High	Medium	Build the master district lookup on Day 1–2 and test joins
R4	GeoJSON missing or heavy	Low	Medium	Download early; simplify geometry; cache in repo
R5	Integration issues between Express and Python	Medium	High	Agree JSON contract Day 5; mock service; end-to-end gate Day 6
R6	Free-tier hosting sleeps or fails during pitch	Medium	High	Warm up before demo; Docker Compose and video backup
R7	Kinyarwanda translations inaccurate	Medium	Medium	Native-speaker review Day 10; keep sentences short

#	Risk	Likelihood	Impact	Mitigation
R8	Scope creep	High	High	Feature freeze Day 10; MoSCoW list; Out-of-scope slide
R9	Team member unavailable	Low	Medium	Pair on critical paths; document everything in `docs/`

---

19. Pitch Plan

19.1 Slide outline (8–10 slides, about 7 minutes plus Q&A)

#	Slide	Key message
1	Title and team	Imbaraga: seeing poverty before the next survey
2	Problem	Survey and Ubudehe cycle gaps leave newly poor households invisible (cite exact figures)
3	Solution	Predict, explain, map, export
4	Live demo	Dashboard → Targeting → Insights
5	How it works	Architecture diagram and data pipeline
6	Model and results	Baseline vs XGBoost, AUC, recall, SHAP drivers, district validation
7	Ethics and fairness	Human decides, privacy by design, fairness checks
8	Scope discipline	What we built vs roadmap (Out-of-scope list as future work)
9	Impact and alignment	NST2, Vision 2050, faster targeting, double-exclusion focus
10	Roadmap and ask	MOUs with LODA/MINALOC, real-time data, SMS/USSD alerts, farmer app, MLOps

19.2 Live demo script (about 3 minutes)

1. **Dashboard:** open the map, toggle to double exclusion, point at flagged districts and the KPI cards.
2. **Targeting:** upload the 200-row demo CSV, press Score All, sort by risk, click the top household to show the three reasons, switch language to Kinyarwanda, download the CSV.
3. **Insights:** show baseline vs XGBoost, global drivers and the fairness chart.
4. State the elapsed time (target: under 1 minute from upload to export).

19.3 Likely judge questions to prepare

- How do you avoid label leakage and overfitting to one region?
- Why XGBoost over logistic regression, and what does the baseline show?
- How would this work with real LODA data and the data protection law?
- How will the model stay current without retraining automation?
- Is it biased against any group, and how did you check?
- Why a decision-support layer and not a payment system?

---

20. Final Readiness Checklist

Product

- [ ] Three pages complete, bilingual labels, all charts C1–C10 render with real data
- [ ] Upload → score → CSV export works with the template file
- [ ] Double-exclusion rule visible on the dashboard

Model

- [ ] Metrics table (accuracy, precision, recall, F1, AUC) for baseline and XGBoost recorded
- [ ] District-grouped cross-validation result recorded
- [ ] Leakage check documented; limitations written down

Engineering

- [ ] README with setup steps; `.env.example` provided
- [ ] All services deployed; Docker Compose fallback tested
- [ ] Tests pass; no secrets in the repo

Pitch

- [ ] Evidence numbers collected with source and year for each
- [ ] Slides finished; two timed rehearsals done
- [ ] Backup video, offline copy and screenshots ready
- [ ] Services warmed 15 minutes before presenting

Appendix A. Shared Types (TypeScript sketch)

```
export type RiskLevel = 'Low' | 'Medium' | 'High';

export interface Driver {
  feature: string;
  value: number | string;
  shap: number;
  direction: 'raises' | 'lowers';
  sentence: { en: string; rw: string };
}

export interface ScoredHousehold {
  id: string;
  districtCode: string;
  probability: number;
  riskLevel: RiskLevel;
  rank: number;
  drivers: Driver[];
}

export interface District {
  code: string;
  nameEn: string;
  nameRw: string;
  povertyRate: number;
  riskScore: number;
  financialAccessRate: number;
  exclusionGap: number;
  doubleExclusion: boolean;
}
```

Appendix B. Items to Confirm in the First 48 Hours

1. Exact EICV round, variable names and microdata access status.
2. FinScope data granularity (district, province or national only) and how to map it to districts.
3. Source and licence of the district GeoJSON.
4. Official poverty line value used for the label.
5. Who reviews Kinyarwanda text.
6. Team roles and daily check-in time.